

Mining Data Streams

4.4 Counting Distinct Elements in a Stream

4.4 Counting Distinct Elements in a Stream

This session covers how to efficiently count unique elements in stream data, and introduces a method for computing approximations using hashing techniques and randomized algorithms for suggested memory usage.

4.4.1 The Count-Distinct Problem

The unique element counting problem seeks to find the number of unique elements in stream data.

► Example

- A website tracks the number of unique users (logged in) per month.
- When a site that receives queries without logging in identifies users by IP address.

1. If the number of unique elements is small, it can be accurately calculated using a hash table or search tree structure.
2. If the number of elements or streams is large, the memory shortage problem occurs.

► How to solve the problem

You can calculate an approximate value by distributing the stream to multiple machines or using an estimation technique that saves memory.

4.4.2 The Flajolet-Martin Algorithm

The Flajolet-Martin Algorithm is a technique that uses hash values to estimate the number of unique elements.

► Generating a hash value

- Hashes each element of the stream to create a bit string
- The length of this bit string must be longer than the number of elements in the entire set, and multiple hash functions are used to generate different hash values

► Discovering unique hash values

Processing various elements results in more hash values, some of which are more likely to have unusual values ending in multiple 0

► the length of a tail

The number of zeros at the end of each hash value is called the 'tail length', and this maximum tail length is set to R . An estimated value for the number of unique elements m is 2^R .

► Probabilistic Analysis

- If m is much greater than 2^R , the probability of finding a hash value with a minimum tail length of r approaches 1
- If m is much less than 2^R , the probability of finding a hash value with a minimum tail length of r approaches 0

► Probabilistic Analysis

Through this analysis, we can conclude that the estimate of m is unlikely to be excessively high or low.

4.4.3 Combining Estimates

► Problem

- When you use the mean, the overestimation greatly affects the mean, making it difficult to obtain an accurate estimate.
- If you use the median, it is difficult to obtain an exact value for the actual number of eigenfactors m because the median always appears as a power of 2.

► Proposed solution

1. Calculate the average of each group by dividing the hash function into smaller groups.
2. Then, take the median of these mean values.

► Effect

This reduces the effect of large values, and if the group is large enough, the mean can be any value, allowing access to the actual m value.

► Group size

The size of each group should be a small multiple of $\log_2 m$.

This approach can obtain a more accurate estimate of the number of unique elements.

4.4.4 Space Requirements

► Need to save

- As you read the stream, you don't need to save the elements you've already seen.
- Store only one integer for each hash function in main memory, keeping only the longest number of zeros recorded by that hash function.

► Single stream processing

If only one stream is processed, millions of hash functions are available, which is much more than enough to obtain the required approximation.

► Multiple stream processing

When multiple streams are processed simultaneously, the main memory can limit the number of hash functions that can be attached to each stream.

► Performance constraints

The real constraint is the time it takes to compute a hash value for each stream element, and the larger the number of hash functions, the more likely they can affect their performance.

Considering memory and time efficiency, it is important to balance the number and performance of hash functions required.

THINK
YOU